VIGNAN'S
FOUNDATION FOR SCIENCE, TECHNOLOGY & RESEARCH
(Deemed to be University) - Estd. u/s 3 of UGC Act 1956

COMPUTER NETWORKS
(25MC410)
I M.C.A II Semester
2025-26 EVEN SEMESTER

Name of the Faculty : Dr. K. Santhi Sri
Associate Professor, Department of CA
VFSTR (Deemed to be University)

Syllabus
Module-2

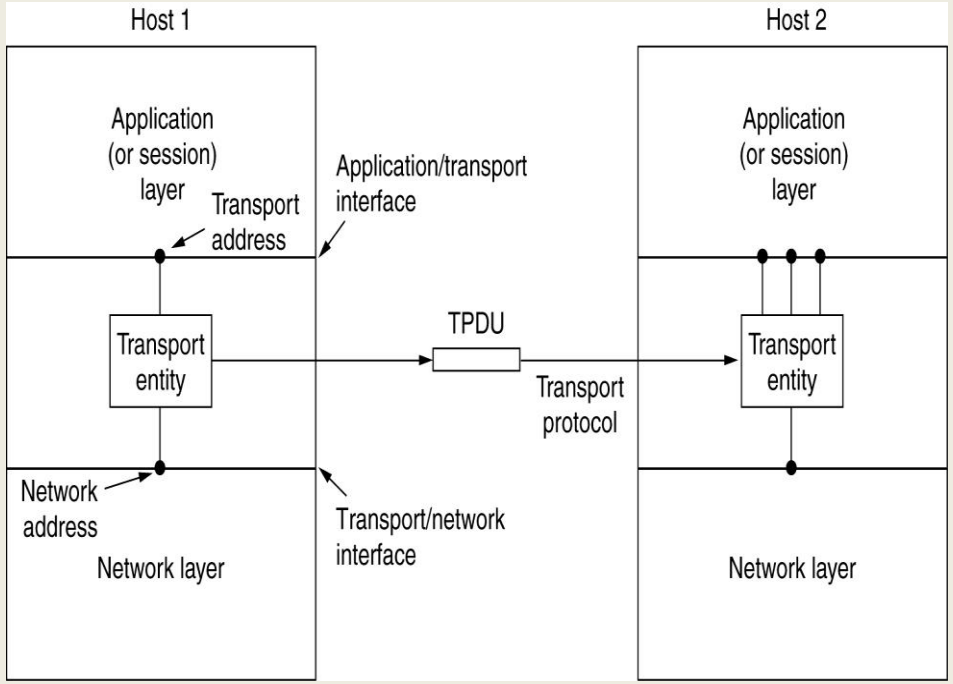
UNIT – 3 **18L+18T+0P+18SL=54 Hours**

TRANSPORT AND APPLICATION LAYER

The Transport Service, Elements of Transport Protocols, Congestion Control, The Internet Transport Protocols: UDP, The Internet Transport Protocols: TCP **Application Layer** : DNS, Electronic Mail, The World Wide Web.

Services Provided to the Upper Layers

- The ultimate goal of the transport layer is to provide efficient, reliable, and cost-effective data transmission service to its users, normally processes in the application layer.
- The software and/or hardware within the transport layer that does the work is called the **transport entity**.
- The transport entity can be located in the operating system kernel, in a library package bound into network applications, in a separate user process, or even on the network interface card.



Services Provided to the Upper Layers

- There are two types of transport service:
- The connection-oriented transport service is similar to the connection-oriented network service in many ways. In both cases, connections have three phases: establishment, data transfer, and release.
- Addressing and flow control are also similar in both layers.
- The connectionless transport service is also very similar to the connectionless network service.
- However, it can be difficult to provide a connectionless transport service on top of a connection-oriented network service, since it is inefficient to set up a connection to send a single packet and then tear it down immediately afterwards.

Services Provided to the Upper Layers

- The transport code runs entirely on the users' machines, but the network layer mostly runs on the routers, which are operated by the carrier (at least for a wide area network).
- What happens if the network layer offers inadequate service? What if it frequently loses packets?
- The users have no real control over the network layer, so they cannot solve the problem of poor service by using better routers or putting more error handling in the data link layer because they don't own the routers.
- The only possibility is to put on top of the network layer another layer that improves the quality of the service.

Transport Service Primitives

- To allow users to access the transport service, the transport layer must provide some operations to application programs, that is, a transport service interface.
- Each transport service has its own interface.
- The transport service is similar to the network service, but there are also some important differences.
- The main difference is that the network service is intended to model the service offered by real networks.
- Real networks can lose packets, so the network service is generally unreliable.
- The connection-oriented transport service, in contrast, is reliable. Real networks are not error-free, but that is precisely the purpose of the transport layer—to provide a reliable service on top of an unreliable network.

Transport Service Primitives

- A second difference between the network service and transport service is whom the services are intended for.
- The network service is used only by the transport entities. Few users write their own transport entities, and thus few users or programs ever see the bare network service.
- In contrast, many programs (and thus programmers) see the transport primitives.
- Consequently, the transport service must be convenient and easy to use.
- To get an idea of what a transport service might be like, consider the five primitives listed.

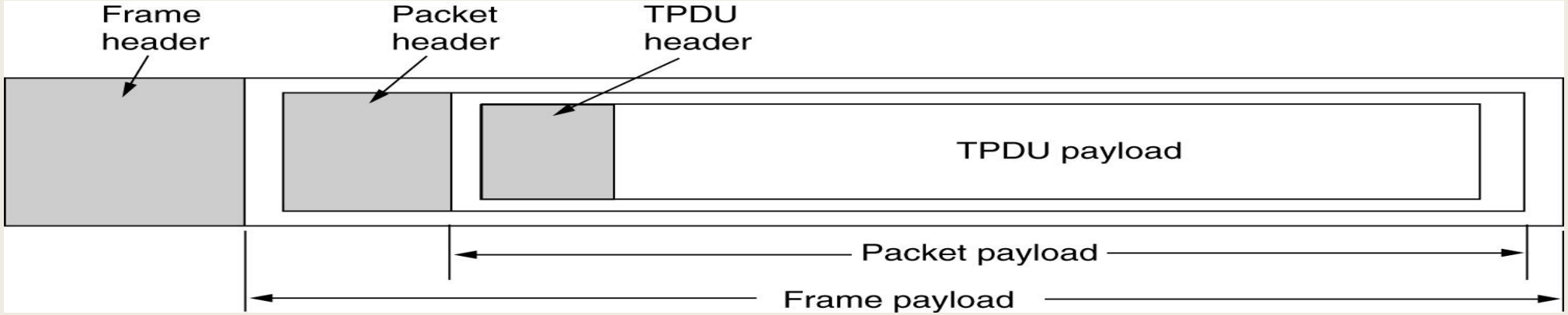
Transport Service Primitives

Primitive	Packet sent	Meaning
LISTEN	(none)	Block until some process tries to connect
CONNECT	CONNECTION REQ.	Actively attempt to establish a connection
SEND	DATA	Send information
RECEIVE	(none)	Block until a DATA packet arrives
DISCONNECT	DISCONNECTION REQ.	This side wants to release the connection

The primitives for a simple transport service.

Transport Service Primitives

- Segments (exchanged by the transport layer) are contained in packets (exchanged by the network layer). In turn, these packets are contained in frames (exchanged by the data link layer).
- When a frame arrives, the data link layer processes the frame header and, if the destination address matches for local delivery, passes the contents of the frame payload field up to the network entity.
- The network entity similarly processes the packet header and then passes the contents of the packet payload up to the transport entity.



Elements of Transport Protocols:

Addressing:

- When an application process wishes to set up a connection to a remote application process, it must specify which process on the remote endpoint to connect to.
- The method normally used is to define transport addresses to which processes can listen for connection requests.
- In the Internet, these endpoints are called ports.
- We will use the generic term TSAP (Transport Service Access Point) to mean a specific endpoint in the transport layer.
- Endpoints in the network layer (i.e., network layer addresses) are called NSAPs

Elements of Transport Protocols:

Addressing:

- The relationship between the NSAPs, the TSAPs, and a transport connection using them.
- Application processes, both clients and servers, can attach themselves to a local TSAP to establish a connection to a remote TSAP.
- The purpose of having TSAPs is that in some networks, each computer has a single NSAP, so some way is needed to distinguish multiple transport endpoints that share that NSAP.

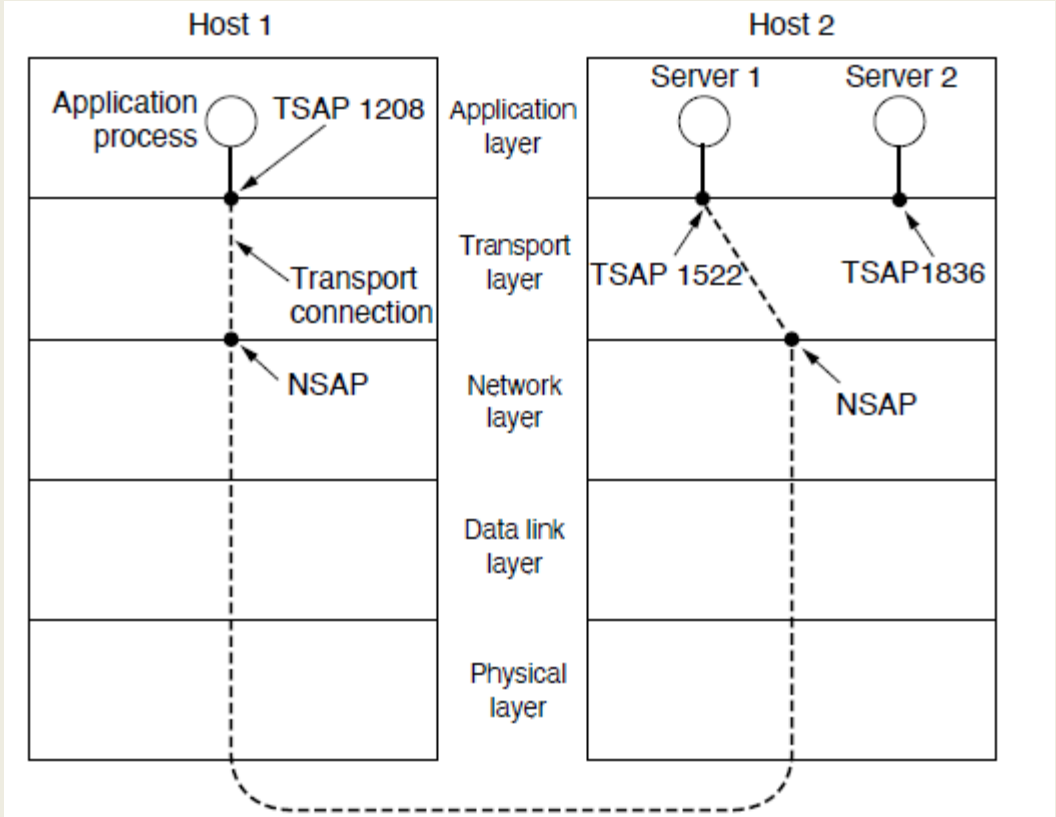


Figure 6-8. TSAPs, NSAPs, and transport connections.

Elements of Transport Protocols:

Connection Establishment:

- Establishing a connection sounds easy, but it is actually tricky.
- At first it would seem sufficient for one transport entity to just send a CONNECTION REQUEST segment to the destination and wait for a CONNECTION ACCEPTED reply.
- The problem occurs when the network can lose, delay, corrupt, and duplicate packets.

Elements of Transport Protocols:

Three-Way Handshake:

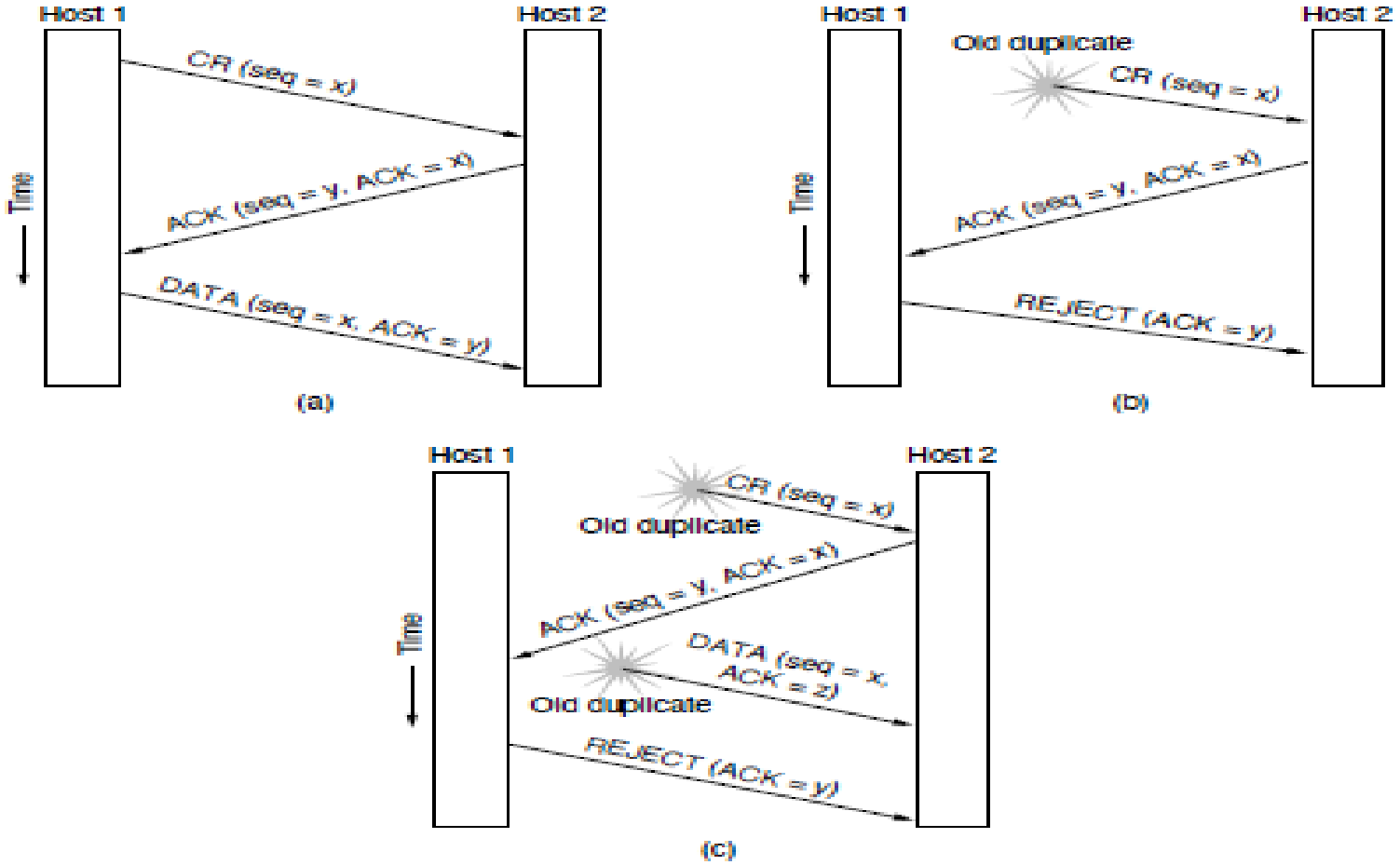


Figure 6-11. Three protocol scenarios for establishing a connection using a three-way handshake. CR denotes CONNECTION REQUEST. (a) Normal operation. (b) Old duplicate CONNECTION REQUEST appearing out of nowhere. (c) Duplicate CONNECTION REQUEST and duplicate ACK.

Elements of Transport Protocols:

Three-Way Handshake:

- This establishment protocol involves one peer checking with the other that the connection request is indeed current.
- The normal setup procedure when host 1 initiates is shown in Fig. 6-11(a).
- Host 1 chooses a sequence number, x, and sends a CONNECTION REQUEST segment containing it to host 2.
- Host 2 replies with an ACK segment acknowledging x and announcing its own initial sequence number, y.
- Finally, host 1 acknowledges host 2's choice of an initial sequence number in the first data segment that it sends.

Elements of Transport Protocols:

Three-Way Handshake:

- In Fig. 6-11(b), the first segment is a delayed duplicate CONNECTION REQUEST from an old connection.
- This segment arrives at host 2 without host 1's knowledge.
- Host 2 reacts to this segment by sending host 1 an ACK segment, in effect asking for verification that host 1 was indeed trying to set up a new connection.
- When host 1 rejects host 2's attempt to establish a connection, host 2 realizes that it was tricked by a delayed duplicate and abandons the connection.

Elements of Transport Protocols:

Three-Way Handshake:

- The worst case is when both a delayed CONNECTION REQUEST and an ACK are floating around in the subnet.
- As in the previous example, host 2 gets a delayed CONNECTION REQUEST and replies to it.
- At this point, it is crucial to realize that host 2 has proposed using y as the initial sequence number for host 2 to host 1 traffic, knowing full well that no segments containing sequence number y or acknowledgements to y are still in existence.
- When the second delayed segment finally arrives at host 2, the fact that z has been acknowledged rather than y tells host 2 that this, too, is an old duplicate.

Elements of Transport Protocols:

Connection Release:

- There are two styles of terminating a connection: asymmetric release and symmetric release.
- Asymmetric release is the way the telephone system works: when one party hangs up, the connection is broken.
- Symmetric release treats the connection as two separate unidirectional connections and requires each one to be released separately.
- Asymmetric release is abrupt and may result in data loss.

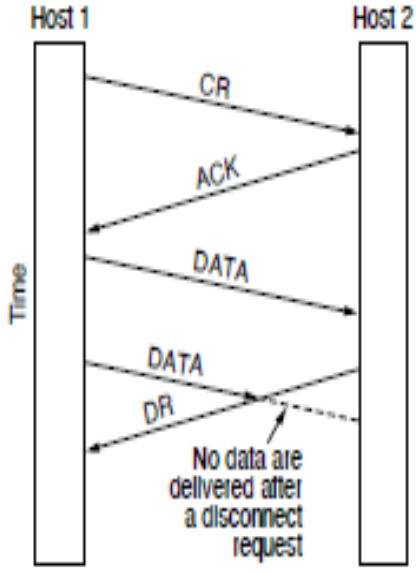


Figure 6-12. Abrupt disconnection with loss of data.

Elements of Transport Protocols:

Connection Release:

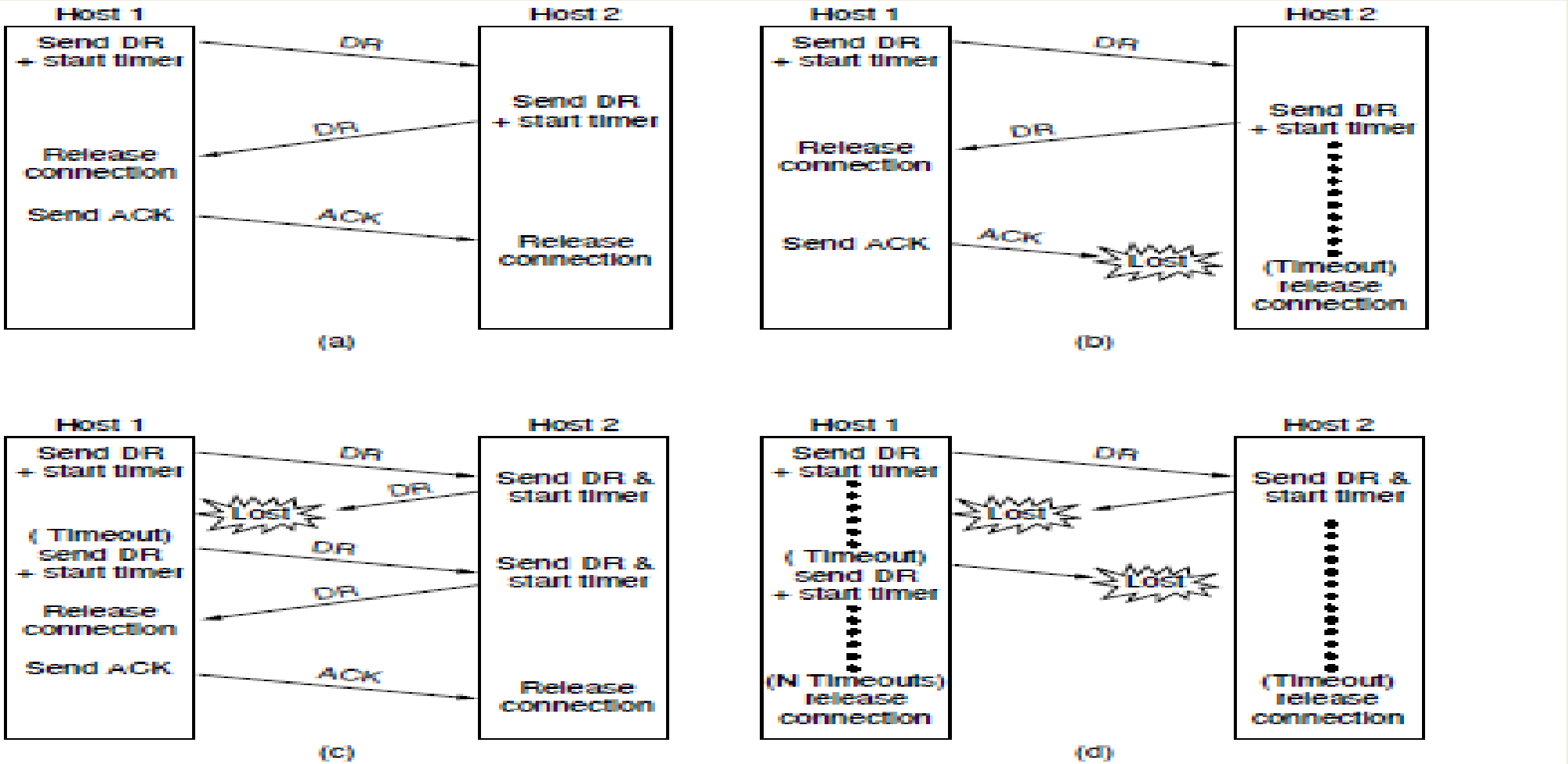


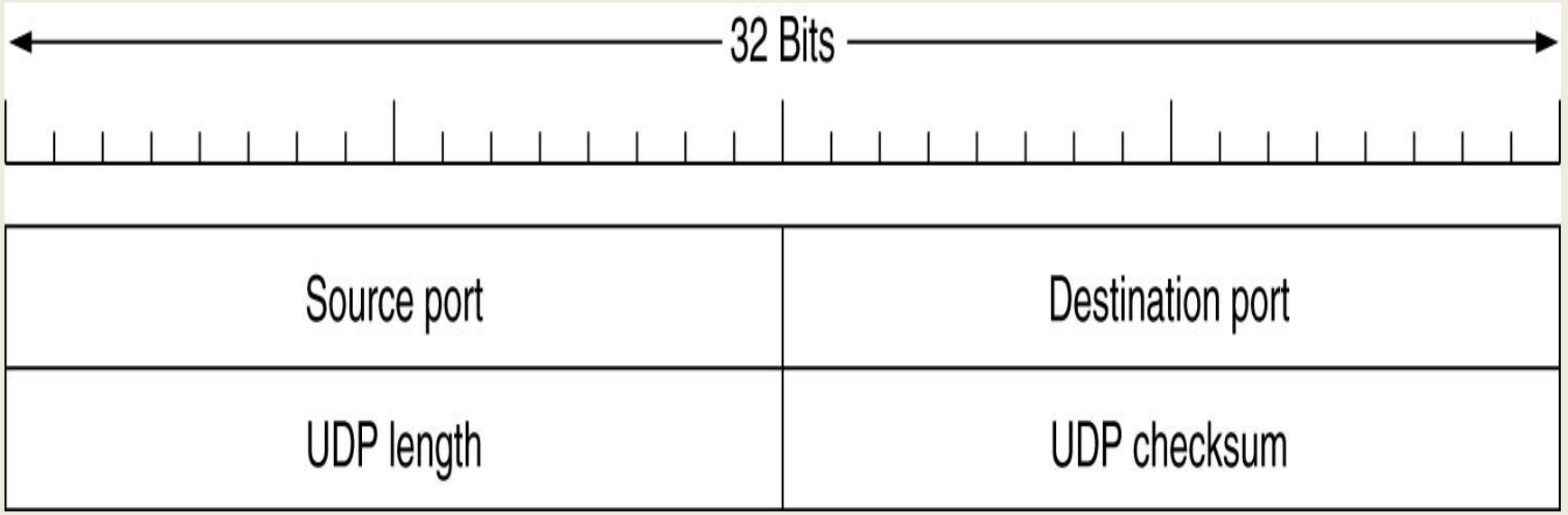
Figure 6-14. Four protocol scenarios for releasing a connection. (a) Normal case of three-way handshake. (b) Final ACK lost. (c) Response lost. (d) Response lost and subsequent DRs lost.

The Internet Transport Protocols: UDP

- The Internet protocol suite supports a connectionless transport protocol called **UDP** (**User Datagram Protocol**).
- UDP provides a way for applications to send encapsulated IP datagrams without having to establish a connection.
- UDP transmits **segments** consisting of an 8-byte header followed by the payload.
- The two **ports** serve to identify the endpoints within the source and destination machines.

Introduction to UDP

The UDP header.



The Internet Transport Protocols: UDP

- When a UDP packet arrives, its payload is handed to the process attached to the destination port. This attachment occurs when the BIND primitive or something similar is used.
- The source port is primarily needed when a reply must be sent back to the source. By copying the *Source port* field from the incoming segment into the *Destination port* field of the outgoing segment, the process sending the reply can specify which process on the sending machine is to get it.

The Internet Transport Protocols: UDP

- The *UDP length* field includes the 8-byte header and the data. The minimum length is 8 bytes, to cover the header. The maximum length is 65,515 bytes, which is lower than the largest number that will fit in 16 bits because of the size limit on IP packets.
- An optional *Checksum* is also provided for extra reliability. It checksums the header, the data, and a conceptual IP pseudo header. When performing this computation, the *Checksum* field is set to zero and the data field is padded out with an additional zero byte if its length is an odd number.

The Internet Transport Protocols: UDP

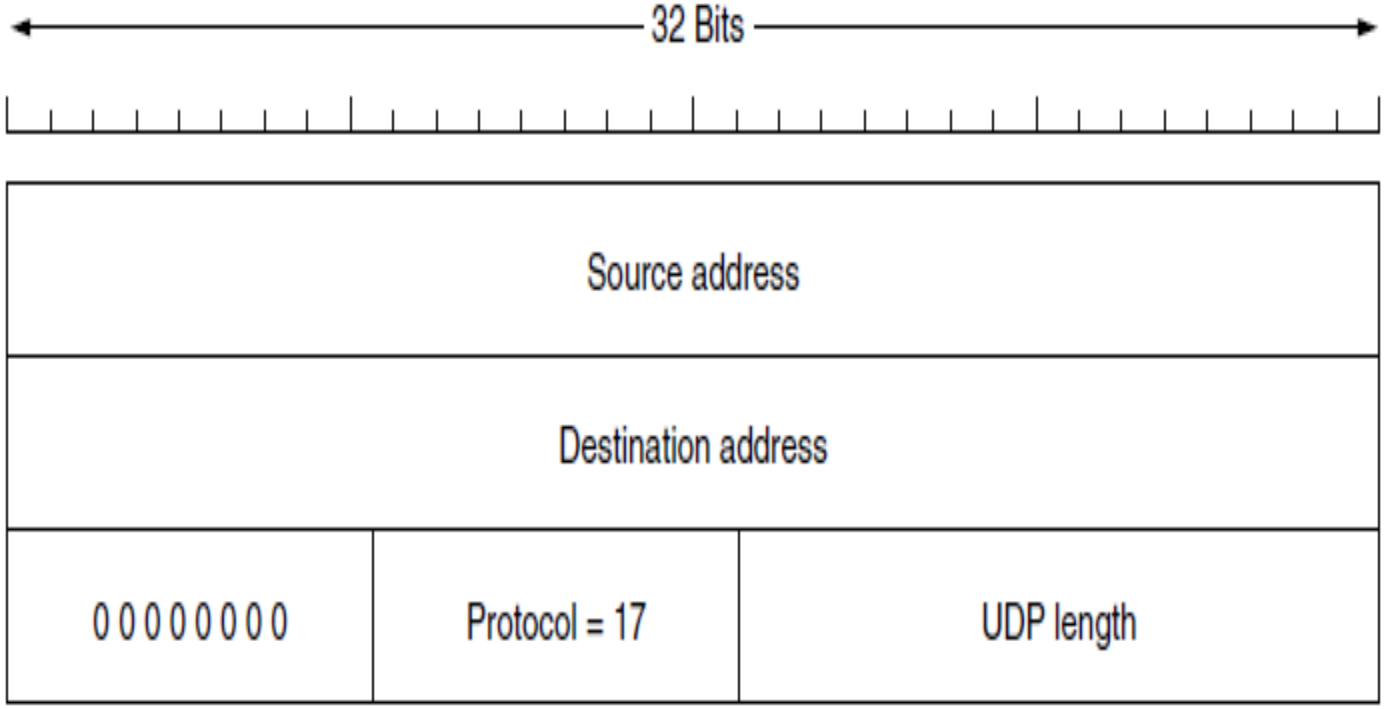


Figure 6-28. The IPv4 pseudoheader included in the UDP checksum.

The Internet Transport Protocols: UDP

- The pseudo header for the case of IPv4 is shown in Fig. 6-28. It contains the 32-bit IPv4 addresses of the source and destination machines, the protocol number for UDP (17), and the byte count for the UDP segment including the header.
- Including the pseudoheader in the UDP checksum computation helps detect misdelivered packets, but including it also violates the protocol hierarchy since the IP addresses in it belong to the IP layer, not to the UDP layer. TCP uses the same pseudoheader for its checksum.

The Internet Transport Protocols: UDP

- It is probably worth mentioning explicitly some of the things that UDP does *not* do.
- It does not do flow control, congestion control, or retransmission upon receipt of a bad segment.
- All of that is up to the user processes.
- What it does do is provide an interface to the IP protocol with the added feature of demultiplexing multiple processes using the ports and optional end-to-end error detection.

The Internet Transport Protocols: TCP

- Introduction to TCP
- The TCP Service Model
- The TCP Protocol
- The TCP Segment Header
- TCP Connection Establishment
- TCP Connection Release

Introduction to TCP

- **TCP (Transmission Control Protocol)** was specifically designed to provide a reliable end-to-end byte stream over an unreliable internetwork.
- An internetwork differs from a single network because different parts may have wildly different topologies, bandwidths, delays, packet sizes, and other parameters.
- Each machine supporting TCP has a TCP transport entity, either a library procedure, a user process, or most commonly part of the kernel.
- In all cases, it manages TCP streams and interfaces to the IP layer. A TCP entity accepts user data streams from local processes, breaks them up into pieces not exceeding 64 KB, and sends each piece as a separate IP datagram.

Introduction to TCP

- The IP layer gives no guarantee that datagrams will be delivered properly, nor any indication of how fast datagrams may be sent.
- It is up to TCP to send datagrams fast enough to make use of the capacity but not cause congestion, and to time out and retransmit any datagrams that are not delivered.
- Datagrams that do arrive may well do so in the wrong order; it is also up to TCP to reassemble them into messages in the proper sequence.
- In short, TCP must furnish good performance with the reliability that most applications want and that IP does not provide.

The TCP Service Model

- TCP service is obtained by both the sender and the receiver creating end points, called **sockets.**
- **Each socket has a socket number** (address) consisting of the IP address of the host and a 16-bit number local to that host, called a **port. A port is the TCP name for a TSAP.**
- **For TCP service to** be obtained, a connection must be explicitly established between a socket on one machine and a socket on another machine.
- A socket may be used for multiple connections at the same time. In other words, two or more connections may terminate at the same socket.

The TCP Service Model

- Connections are identified by the socket identifiers at both ends, that is, (*socket1*, *socket2).*
- *No* virtual circuit numbers or other identifiers are used.
- Port numbers below 1024 are reserved for standard services that can usually only be started by privileged users. They are called **well-known ports.**
- For example, any process wishing to remotely retrieve mail from a host can connect to the destination host's port 143 to contact its IMAP daemon.

The TCP Service Model

Some assigned ports.

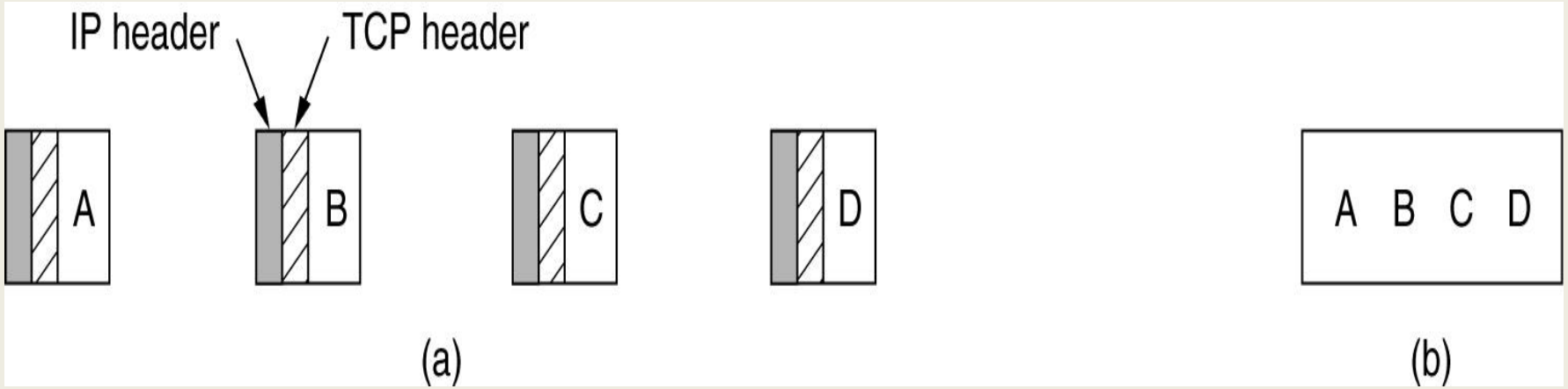
Port	Protocol	Use
21	FTP	File transfer
23	Telnet	Remote login
25	SMTP	E-mail
69	TFTP	Trivial File Transfer Protocol
79	Finger	Lookup info about a user
80	HTTP	World Wide Web
110	POP-3	Remote e-mail access
119	NNTP	USENET news

The TCP Service Model

- All TCP connections are full duplex and point-to-point. Full duplex means that traffic can go in both directions at the same time.
- Point-to-point means that each connection has exactly two end points. TCP does not support multicasting or broadcasting.
- A TCP connection is a byte stream, not a message stream. Message boundaries are not preserved end to end.
- For example, if the sending process does four 512-byte writes to a TCP stream, these data may be delivered to the receiving process as four 512-byte chunks, two 1024-byte chunks, one 2048-byte chunk, or some other way.

The TCP Service Model

- There is no way for the receiver to detect the unit(s) in which the data were written, no matter how hard it tries.



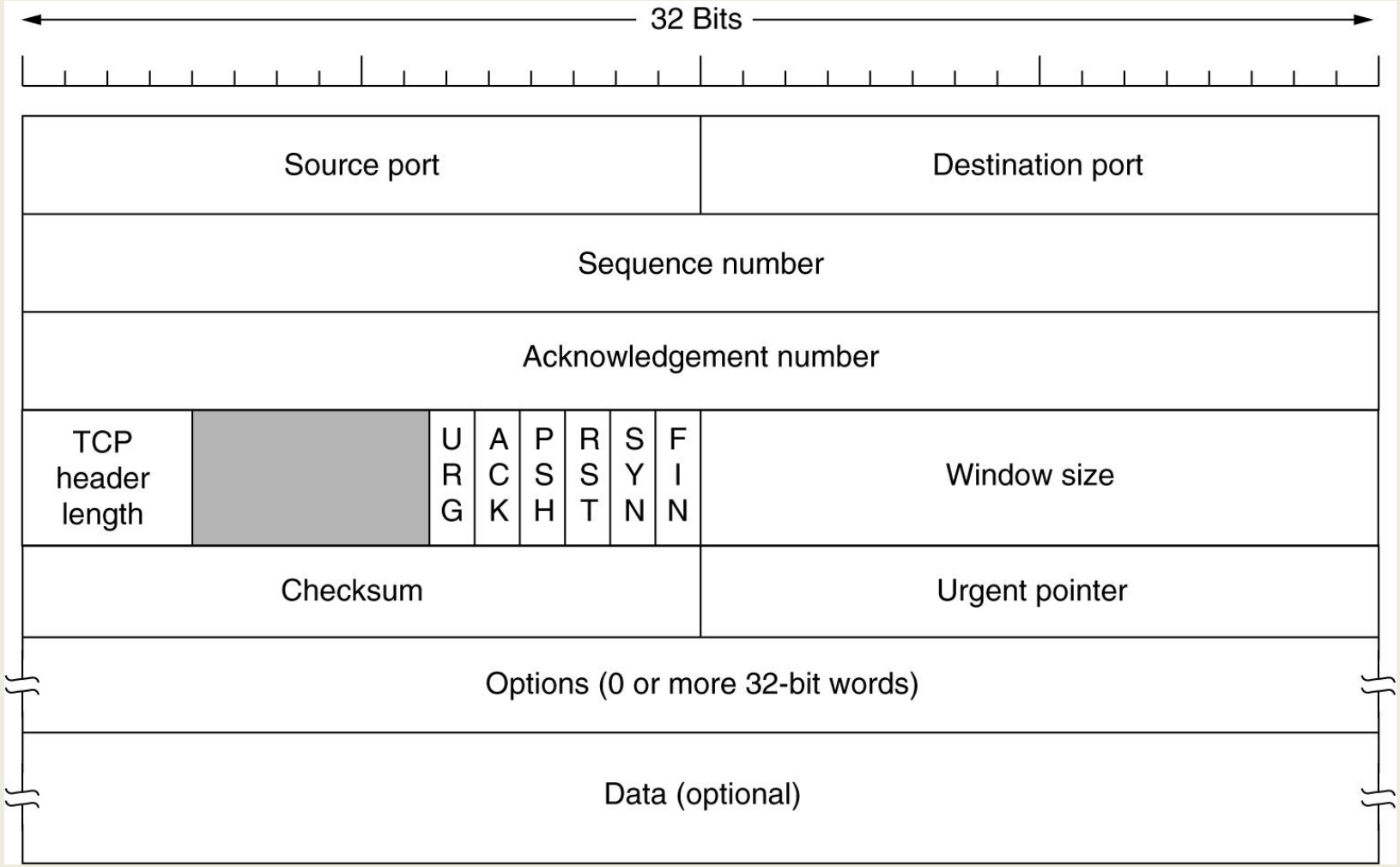
(a) Four 512-byte segments sent as separate IP datagrams.

(b) The 2048 bytes of data delivered to the application in a single READ CALL.

The TCP Protocol

- The sending and receiving TCP entities exchange data in the form of segments.
- A **TCP segment consists of a fixed 20-byte header (plus an optional part)** followed by zero or more data bytes.
- The TCP software decides how big segments should be. It can accumulate data from several writes into one segment or can split data from one write over multiple segments.
- Two limits restrict the segment size.
- First, each segment, including the TCP header, must fit in the 65,515- byte IP payload.
- Second, each link has an **MTU (Maximum Transfer Unit).**

The TCP Segment Header



TCP Header

The TCP Segment Header

- Every segment begins with a fixed-format, 20-byte header. The fixed header may be followed by header options.
- After the options, if any, up to 65,535 − 20 − 20 = 65,495 data bytes may follow, where the first 20 refer to the IP header and the second to the TCP header.
- Segments without any data are legal and are commonly used for acknowledgements and control messages.
- The *Source port and Destination port fields identify the local end points of the connection. A TCP port plus its* host's IP address forms a 48-bit unique end point. The source and destination end points together identify the connection.

The TCP Segment Header

- This connection identifier is called a **5 tuple because it consists of five pieces of information: the protocol (TCP), source** IP and source port, and destination IP and destination port.
- The *Sequence number and Acknowledgement number fields perform their* usual functions.
- It is a **cumulative acknowledgement because it** summarizes the received data with a single number. It does not go beyond lost data. Both are 32 bits because every byte of data is numbered in a TCP stream.
- The *TCP header length tells how many 32-bit words are contained in the TCP* header. This information is needed because the *Options field is of variable length*, so the header is, too.

The TCP Segment Header

- *URG is set to 1 if the Urgent pointer is in use. The Urgent pointer is used to* indicate a byte offset from the current sequence number at which urgent data are to be found. This facility is in lieu of interrupt messages.
- The *ACK bit is set to 1 to indicate that the Acknowledgement number is valid.* This is the case for nearly all packets. If *ACK is 0, the segment does not contain* an acknowledgement, so the *Acknowledgement number field is ignored.*
- The *PSH bit indicates PUSHed data. The receiver is hereby kindly requested* to deliver the data to the application upon arrival and not buffer it until a full buffer has been received.

The TCP Segment Header

- The *RST bit is used to abruptly reset a connection that has become confused* due to a host crash or some other reason. It is also used to reject an invalid segment or refuse an attempt to open a connection. In general, if you get a segment with the *RST bit on, you have a problem on your hands.*
- The *SYN bit is used to establish connections. The connection request has SYN = 1 and ACK = 0 to indicate that the piggyback acknowledgement field is not* in use. The connection reply does bear an acknowledgement, however, so it has *SYN = 1 and ACK = 1.*

The TCP Segment Header

- The *FIN bit is used to release a connection. It specifies that the sender has no* more data to *transmit. After closing a connection, the closing process* may continue to *receive data indefinitely. Both SYN and FIN segments have sequence* numbers and are thus guaranteed to be processed in the correct order.
- Flow control in TCP is handled using a variable-sized sliding window. The *Window size field tells how many bytes may be sent starting at the byte acknowledged.* A *Window size field of 0 is legal and says that the bytes up to and including Acknowledgement number − 1 have been received.*

The TCP Segment Header

- A *Checksum is also provided for extra reliability. It checksums the header,* the data, and a conceptual pseudo header in exactly the same way as UDP, except that the pseudo header has the protocol number for TCP (6) and the checksum is mandatory.
- The *Options field provides a way to add extra facilities not covered by the* regular header. The options are of variable length, fill a multiple of 32 bits by using padding with zeros, and may extend to 40 bytes to accommodate the longest TCP header that can be specified.

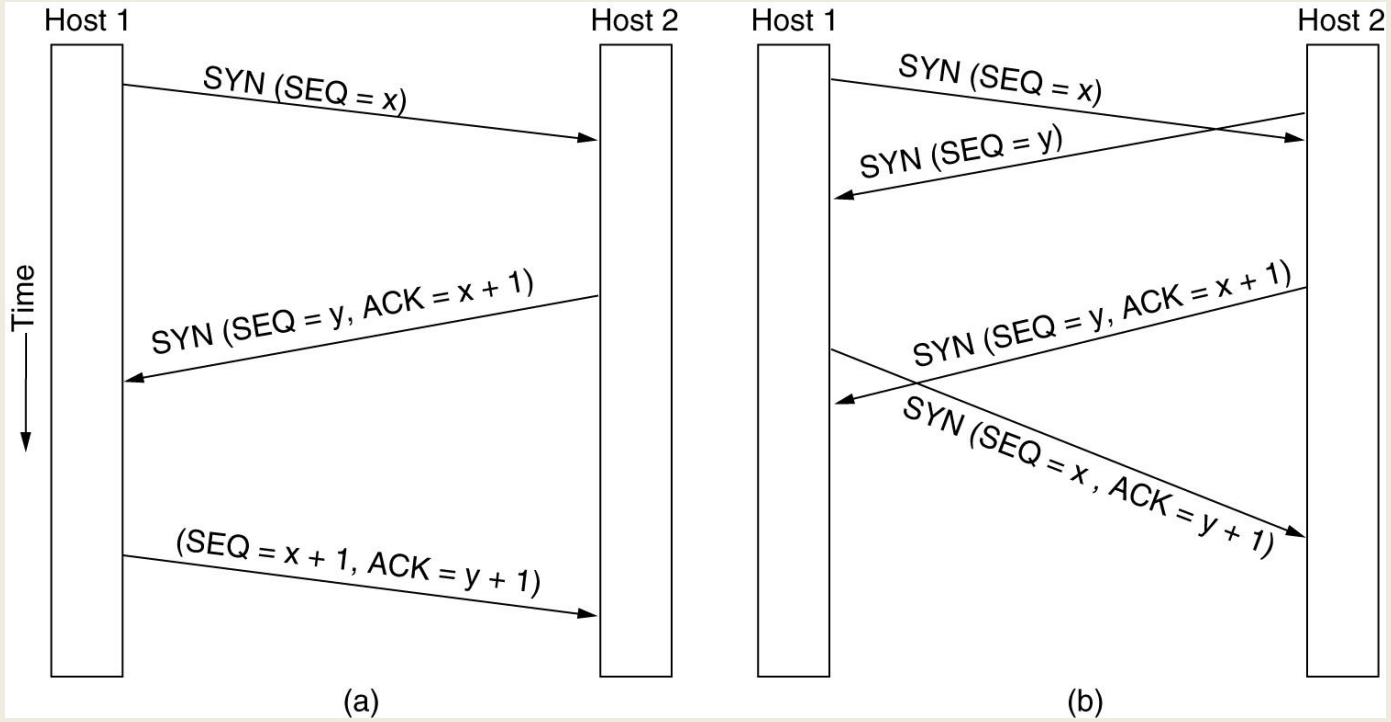
TCP Connection Establishment

- Connections are established in TCP by means of the three-way handshake.
- To establish a connection, one side, say, the server, passively waits for an incoming connection by executing the LISTEN and ACCEPT primitives in that order, either specifying a specific source or nobody in particular.
- The other side client, executes a CONNECT primitive, specifying the IP address and port to which it wants to connect, the maximum TCP segment size it is willing to accept, and optionally some user data. The CONNECT primitive sends a TCP segment with the *SYN bit on and ACK bit off and* waits for a response.

TCP Connection Establishment

- When this segment arrives at the destination, the TCP entity there checks to see if there is a process that has done a LISTEN on the port given in the *Destination port field. If not, it sends a reply with the RST bit on to reject the connection.* I
- f some process is listening to the port, that process is given the incoming TCP segment. It can either accept or reject the connection. If it accepts, an acknowledgement segment is sent back.

TCP Connection Establishment



(a) TCP connection establishment in the normal case.

(b) Call collision.

TCP Connection Establishment

- In the event that two hosts simultaneously attempt to establish a connection between the same two sockets, the sequence of events is as illustrated in Fig. 6-37(b).
- The result of these events is that just one connection is established, not two, because connections are identified by their end points. If the first setup results in a connection identified by (*x*, *y*) *and the second one does too*, *only one* table entry is made, namely, for (*x*, *y*).

TCP Connection Release

- Although TCP connections are full duplex, to understand how connections are released it is best to think of them as a pair of simplex connections.
- Each simplex connection is released independently of its sibling. To release a connection, either party can send a TCP segment with the *FIN bit set, which means that it has no* more data to transmit. When the *FIN is acknowledged, that direction is shut down* for new data. Data may continue to flow indefinitely in the other direction, however. When both directions have been shut down, the connection is released. Normally, four TCP segments are needed to release a connection: one *FIN and* one *ACK for each direction.*

The Domain Name System(DNS)

- An organization's Web server could thus be referred to as www.vignanuniversity.org regardless of its IP address.
- Because the devices along a network path forward traffic to its destination based on IP address, these human-readable domain names must be converted to IP addresses; the DNS (Domain Name System) is the mechanism that does so.
- DNS is a hierarchical naming scheme and a distributed database system that implements this naming scheme. It is primarily used for mapping host names to IP addresses

The Domain Name System(DNS)

- To map a name onto an IP address, an application program calls a library procedure, passing this function the name as a parameter.
- This process is sometimes referred to as the stub resolver.
- The stub resolver sends a query containing the name to a local DNS resolver, often called the local recursive resolver or simply the local resolver, which subsequently performs a so-called recursive lookup for the name against a set of DNS resolvers.
- The local recursive resolver ultimately returns a response with the corresponding IP address to the stub resolver, which then passes the result to the function that issued the query in the first place.

The DNS Name Space and Hierarchy:

- For the Internet, the top of the naming hierarchy is managed by an organization called ICANN (Internet Corporation for Assigned Names and Numbers).
- The Internet is divided into over 250 **top-level domains**, where each domain covers many hosts.
- Each domain is partitioned into subdomains, and these are further partitioned, and so on.
- The leaves of the tree represent domains that have no subdomains (but do contain machines).
- A leaf domain may contain a single host, or it may represent a company and contain thousands of hosts.

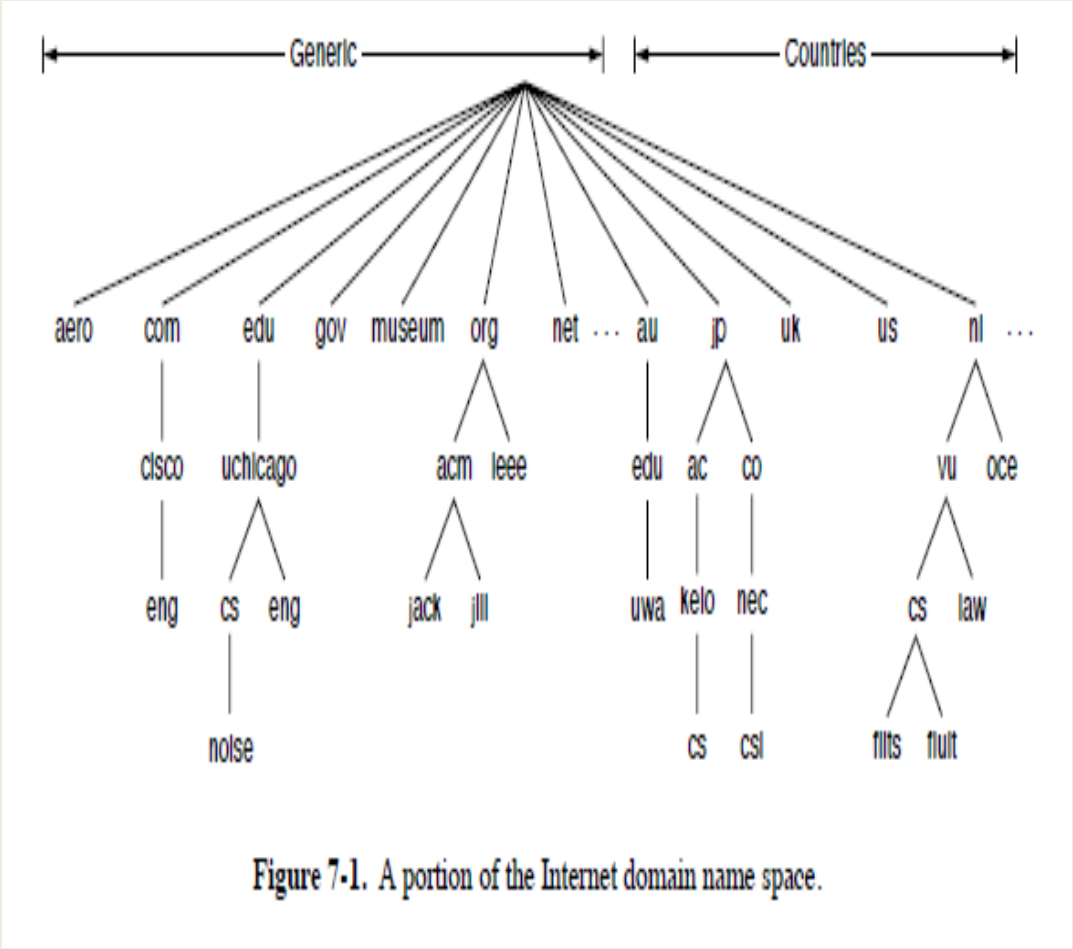


Figure 7-1. A portion of the Internet domain name space.

DNS Queries and Responses:

DNS Queries:

- A DNS client typically issues a query to a local recursive resolver, which performs an iterative query to ultimately resolve the query.
- The most common query type is an A record query, which asks for a mapping from a domain name to an IP address for a corresponding Internet endpoint.

DNS Responses and Resource Records:

- Every domain, whether it is a single host or a top-level domain, can have a set of resource records associated with it. These records are the DNS database.
- For a single host, the most common resource record is just its IP address, but many other kinds of resource records also exist.
- When a resolver gives a domain name to DNS, what it gets back are the resource records associated with that name.

DNS Queries and Responses:

DNS Responses and Resource Records:

- A resource record is a five-tuple.

  Domain_name Time_to_live Class Type Value

- The Domain name tells the domain to which this record applies. This field is thus the primary search key used to satisfy queries.
- The Time to live field gives an indication of how stable the record is. Information that is highly stable is assigned a large value, such as 86400. Information that is volatile or that operators may want to change frequently may be assigned a small value, such as 60 seconds.
- The third field of every resource record is the Class. For Internet information, it is always IN. For non-Internet information, other codes can be used.

DNS Queries and Responses:

DNS Responses and Resource Records:

- The Type field tells what kind of record this is. There are many kinds of DNS records.

Type	Meaning	Value
SOA	Start of authority	Parameters for this zone
A	IPv4 address of a host	32-Bit integer
AAAA	IPv6 address of a host	128-Bit integer
MX	Mail exchange	Priority, domain willing to accept email
NS	Name server	Name of a server for this domain
CNAME	Canonical name	Domain name
PTR	Pointer	Alias for an IP address
SPF	Sender policy framework	Text encoding of mail sending policy
SRV	Service	Host that provides it
TXT	Text	Descriptive ASCII text

Figure 7-4. The principal DNS resource record types.

DNS Queries and Responses:

DNS Zones:

- To avoid the problems associated with having only a single source of information, the DNS name space is divided into nonoverlapping zones.
- Where the zone boundaries are placed within a zone is up to that zone's administrator.
- This decision is made in large part based on how many name servers are desired, and where.

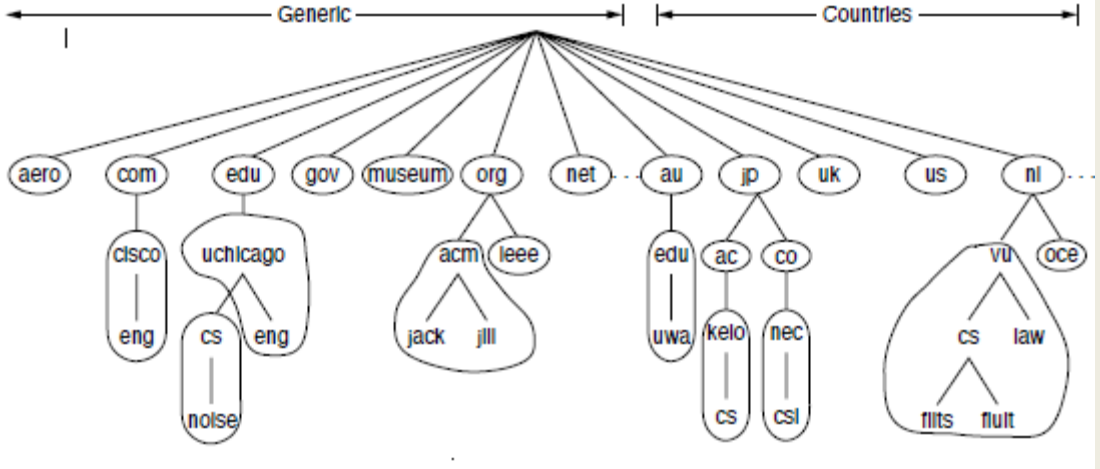


Figure 7-6. Part of the DNS name space divided into zones (which are circled).

DNS Queries and Responses:

Name Resolution:

- Each zone is associated with one or more name servers.
- These are hosts that hold the database for the zone.
- Normally, a zone will have one primary name server, which gets its information from a file on its disk, and one or more secondary name servers, which get their information from the primary name server.
- The process of looking up a name and finding an address is called name resolution.
- When a resolver has a query about a domain name, it passes the query to a local name server.
- If the domain being sought falls under the jurisdiction of the name server, such as top.cs.vu.nl falling under cs.vu.nl, it returns the authoritative resource records.

DNS Queries and Responses:

Name Resolution:

- An authoritative record is one that comes from the authority that manages the record and is thus always correct.
- Step 1 shows the query that is sent to the local name server.
- The query contains the domain name sought, the type (A), and the class(IN).
- The next step is to start at the top of the name hierarchy by asking one of the root name servers. These name servers have information about each top-level domain.

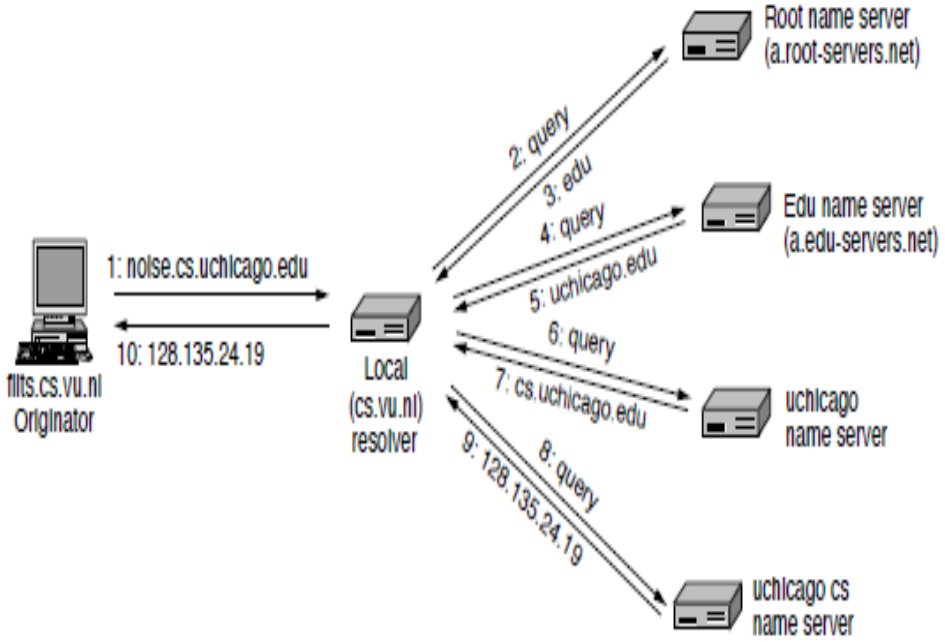


Figure 7-7. Example of a resolver looking up a remote name in 10 steps.